Lập trình mạng · Chương 2

TCP: kết nối giữa client và server

TS. Đặng Ngọc Hùng

Chương 2: Lập trình socket TCP và đa luồng

1 TCP

- Kết nối giữa client và server
- Timeout, half-close và lỗi kết nối
- Bài toán ứng dụng: web server tối giản trên TCP

2 Đa luồng

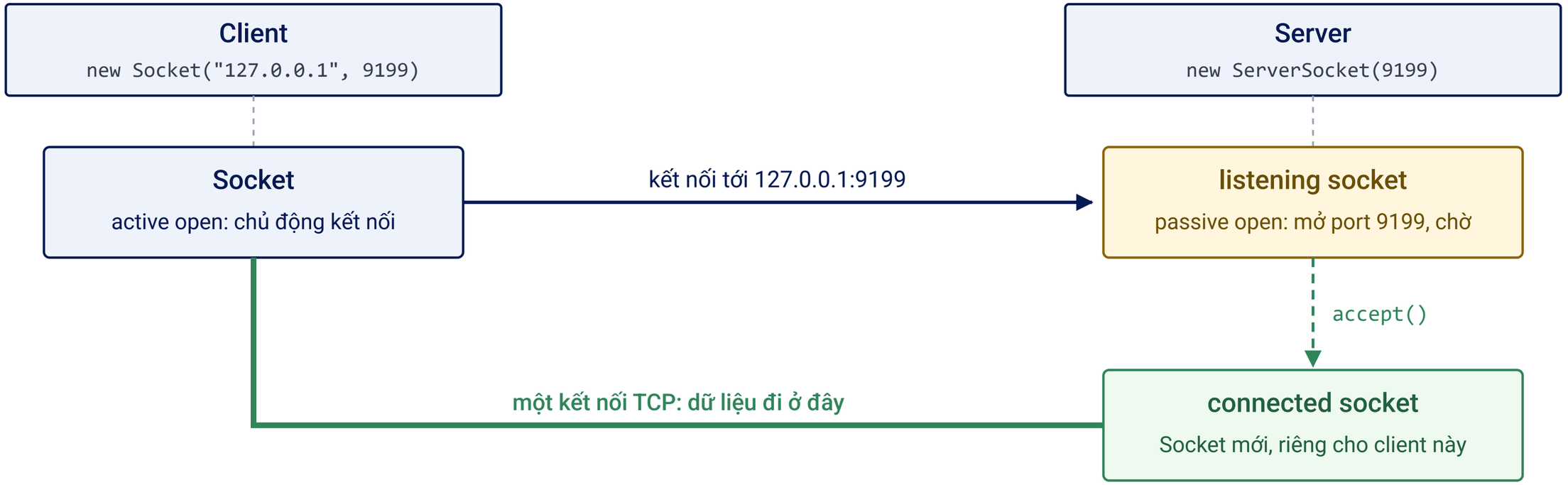
- Thread và race condition
- Synchronized, happens-before và wait/notify
- Executor, thread pool và virtual thread
- Bài toán ứng dụng: web server phục vụ nhiều client

Từ phần này, hai đầu kết nối là chương trình Java

1. Bài toán ứng dụng của Chương 1: nói HTTP bằng tay với **NetCat** — công cụ tự lắng nghe, tự kết nối
2. Từ phần này: server và client là **chương trình Java**, phải tự mở port và tự kết nối

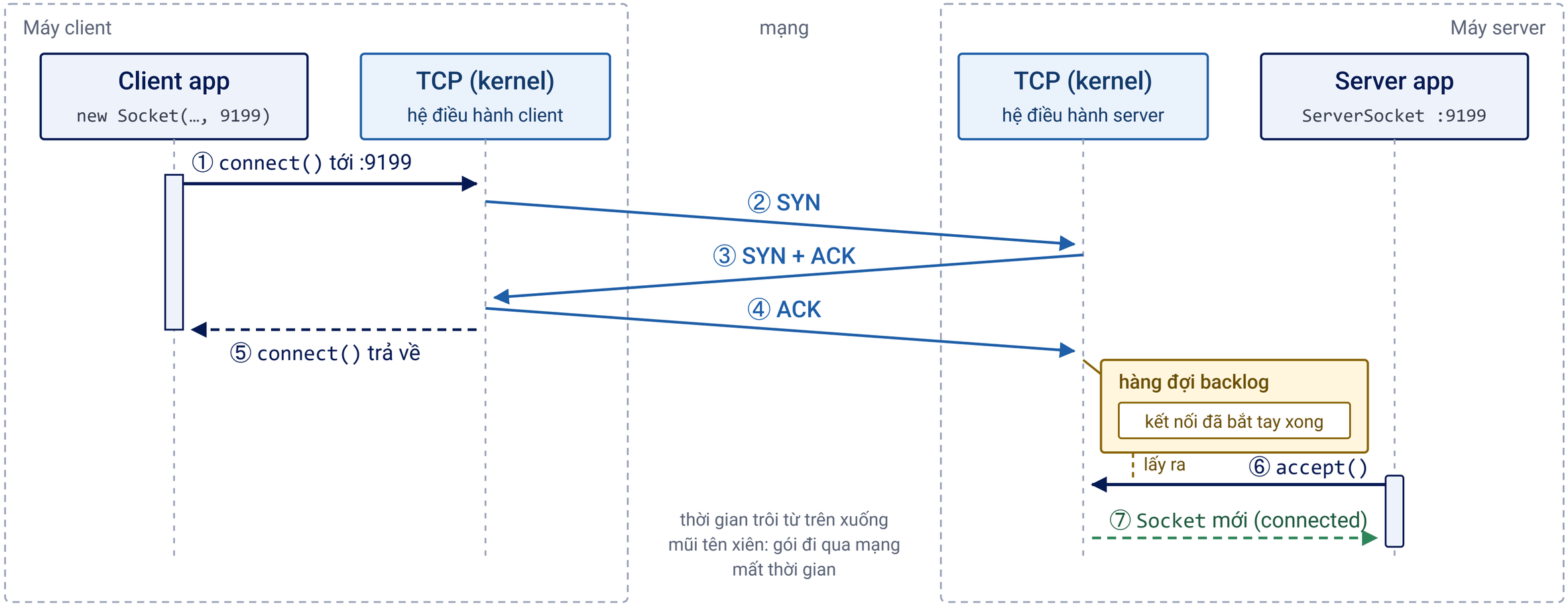
> **Câu hỏi của phần:** hai chương trình Java thiết lập một kết nối TCP như thế nào, và sau đó mỗi bên dùng đối tượng nào để trao đổi dữ liệu?

Server mở port và chờ trước, client chủ động kết nối tới



> Server dùng **hai loại** socket: listening socket (`ServerSocket`) để chờ kết nối, và mỗi kết nối một connected socket (`Socket`) để trao đổi dữ liệu.

Hệ điều hành thực hiện bắt tay, accept() chỉ lấy kết nối đã thiết lập



`connect()` trả về: bắt tay đã xong ở phía client. **Chưa cho biết** server đã gọi `accept()`.

`accept()` không bắt tay. Nó lấy một kết nối đã thiết lập khỏi backlog và trả về connected socket.

Echo server: mở port 9199, chờ một kết nối, gửi lại đúng dòng nhận được

Java

```
try (ServerSocket listener = new ServerSocket(9199)) {  // passive open: mở port, chờ
    System.out.println("listening on port " + listener.getLocalPort());
    try (Socket connected = listener.accept()) {  // lấy một kết nối khỏi hàng đợi
        System.out.println("accepted: local port " + connected.getLocalPort()
                + ", client port " + connected.getPort());
        var in = new BufferedReader(new InputStreamReader(connected.getInputStream(), UTF_8));
        var out = new BufferedWriter(new OutputStreamWriter(connected.getOutputStream(), UTF_8));
        String line = in.readLine();  // đọc một dòng
        out.write(line + "\n");  // gửi lại đúng dòng đó
        out.flush();
    }
}
```

Kết quả chạy

```
listening on port 9199
accepted: local port 9199, client port 59887
```

Câu lệnh new Socket thực hiện active open ở phía client

Java

```
try (Socket socket = new Socket("127.0.0.1", 9199)) {  // active open: kết nối tới server
    System.out.println("connected: local port " + socket.getLocalPort()
            + ", server port " + socket.getPort());
    var out = new BufferedWriter(new OutputStreamWriter(socket.getOutputStream(), UTF_8));
    var in = new BufferedReader(new InputStreamReader(socket.getInputStream(), UTF_8));
    out.write("hello\n");  // gửi một dòng
    out.flush();
    System.out.println("echoed: " + in.readLine());  // đọc dòng server gửi lại
}
```

Kết quả chạy

```
connected: local port 59887, server port 9199
echoed: hello
```

- Cùng một kết nối, nhìn từ hai đầu: local port và remote port **đổi chỗ**

Không có socket lắng nghe ở port đích thì kết nối bị từ chối ngay

Java

```
try (Socket socket = new Socket("127.0.0.1", 9199)) {  // active open: kết nối tới server
```

Kết quả chạy

```
Exception in thread "main" java.net.ConnectException: Connection refused: connect
        at java.base/sun.nio.ch.Net.connect0(Native Method)
        at java.base/sun.nio.ch.Net.connect(Net.java:601)
        at java.base/sun.nio.ch.Net.connect(Net.java:590)
        at java.base/sun.nio.ch.NioSocketImpl.connect(NioSocketImpl.java:583)
        at java.base/java.net.SocksSocketImpl.connect(SocksSocketImpl.java:327)
        at java.base/java.net.Socket.connect(Socket.java:751)
        at java.base/java.net.Socket.connect(Socket.java:686)
        at java.base/java.net.Socket.<init>(Socket.java:555)
        at java.base/java.net.Socket.<init>(Socket.java:324)
        at netlab.echo.EchoClient.main(EchoClient.java:21)
```

- Dòng trên cùng: **loại lỗi** — TCP của máy đích trả RST vì không có socket nào lắng nghe ở port 9199
- Dòng dưới cùng: **mã của mình** — chỗ gọi gây lỗi

Câu hỏi 1

Ở phía client, `connect()` vừa trả về thành công. Điều nào chắc chắn đã xảy ra?

A. Server đã gọi `accept()` và nhận được connected socket.

B. Bắt tay TCP đã xong ở phía client; kết nối có thể chưa được server lấy ra khỏi backlog.

C. Server đã đọc được dòng đầu tiên client gửi.

D. Server đã cấp một port mới riêng cho kết nối này.

Câu hỏi 1

Ở phía client, `connect()` vừa trả về thành công. Điều nào chắc chắn đã xảy ra?

A. Server đã gọi `accept()` và nhận được connected socket.

B. Bắt tay TCP đã xong ở phía client; kết nối có thể chưa được server lấy ra khỏi backlog.

C. Server đã đọc được dòng đầu tiên client gửi.

D. Server đã cấp một port mới riêng cho kết nối này.

Bắt tay do TCP trong hệ điều hành hai máy thực hiện. `connect()` trả về khi client gửi ACK; kết nối vào backlog khi ACK tới server; `accept()` lấy kết nối ra và có thể đến sau. Kết nối dùng chung port 9199 của server, không có port mới.

Câu hỏi 2

Bạn chạy EchoClient khi EchoServer chưa chạy, trên cùng một máy. Điều gì xảy ra?

A. Client chờ đến khi server chạy rồi tự kết nối.

B. Client kết nối được, nhưng `readLine()` trả về null.

C. Client nhận ConnectException (connection refused) gần như ngay lập tức.

D. Client tự chuyển sang một port khác còn trống trên máy đó.

Câu hỏi 2

Bạn chạy EchoClient khi EchoServer chưa chạy, trên cùng một máy. Điều gì xảy ra?

A. Client chờ đến khi server chạy rồi tự kết nối.

B. Client kết nối được, nhưng `readLine()` trả về null.

C. Client nhận ConnectException (connection refused) gần như ngay lập tức.

D. Client tự chuyển sang một port khác còn trống trên máy đó.

Không có socket nào lắng nghe ở port 9199 nên TCP của máy đích trả RST và `connect()` ném ConnectException ngay. TCP không chờ server xuất hiện, và client không tự đổi port đích.

Tổng kết

1. Server chờ bằng **listening socket** (passive open); client chủ động kết nối tới (active open)

2. **Bắt tay ba bước** do hệ điều hành thực hiện; `accept()` lấy kết nối khỏi **backlog**, trả về **connected socket**

3. Không có socket lắng nghe ở port đích → **RST**, connection refused ngay

Từ khóa cần nhớ socket · ServerSocket · active open · passive open · listening socket · connected socket · bắt tay ba bước · backlog · RST · connection refused

Mở rộng

Chủ đề nâng cao

- Tùy chọn SO_REUSEADDR khi khởi động lại server
- Bind address: 0.0.0.0 và 127.0.0.1

Bài tập mở rộng

Sửa EchoServer để in endpoint hai đầu của mỗi kết nối và phục vụ đúng ba client lần lượt rồi dừng.

Tiếp theo

TCP: timeout, half-close và lỗi kết nối